

Reinforcement fine-tuning LLMs with GRPO

🕒 Created	August 18, 2025 5:41 PM
👤 OfferedBy	Deeplearning.ai
👤 Platform	Deeplearning.ai
📅 Date	August 18, 2025
👥 Instructors	Andrew Ng
🛠️ Tools	

Introduction

Reinforcement fine-tuning (RFT) which is a training technique which uses  Reinforcement Learning to improve the performance of LLMs on tasks that require multi-step reasoning like math or code generation.

Reinforcement fine-tuning guides the model to discover solutions on its own rather than relying on pre-existing examples as traditional Supervised Learning

This approach allows the model to adapt with much less training data (couple of dozen examples)

Taken a running example of the playing wordle to guess 5 letter word in less than 6 tries or fewer by prompting a Qwen-7.5B million model, analyze the performance and develop a reward function to help the model learn how to do better overtime. This reward function is the key component of Group Relative Policy Optimization (GRPO) - the learning algorithm developed by Deepseek to carry out  Reinforcement Learning of reasoning tasks

In Group Relative Policy Optimization (GRPO), the LLM produces multiple responses to a single prompt that are scored based on a reward function based on verifiable metrics like correct formatting or functioning encode. This reward function is the key difference between GRPO and other  Reinforcement Learning algorithms like Proximal Policy Optimization (PPO), or DPO that rely on human feedback or complex multi-modal systems to assign rewards

Introduction to Reinforcement Learning

Supervised Fine-Tuning (SFT)

Traditionally we teach LLMs tasks like classification, Named Entity Recognition and Code Generation using Supervised Fine-Tuning (SFT) process. We assemble a labelled dataset with prompt and response pairs. During training, each example goes through two processes

- Forward pass: Model generates an output for a given prompt
- Backward pass: Compare the model's output to the correct response, compute the error and update the weights to reduce the error

For simple tasks, can show math problem and final answer and the model will learn to predict correct answers even for the unseen problems

For more complex problems, we can include reasoning traces in <think> </think> tags along with the <answer></answer>. By structuring the model this way, can teach the model two aspects simultaneously

- Output format - how to use tags thoughts as separate from the final answer
- Ability to do step-by-step reasoning

Limitations of Supervised Fine-Tuning (SFT)

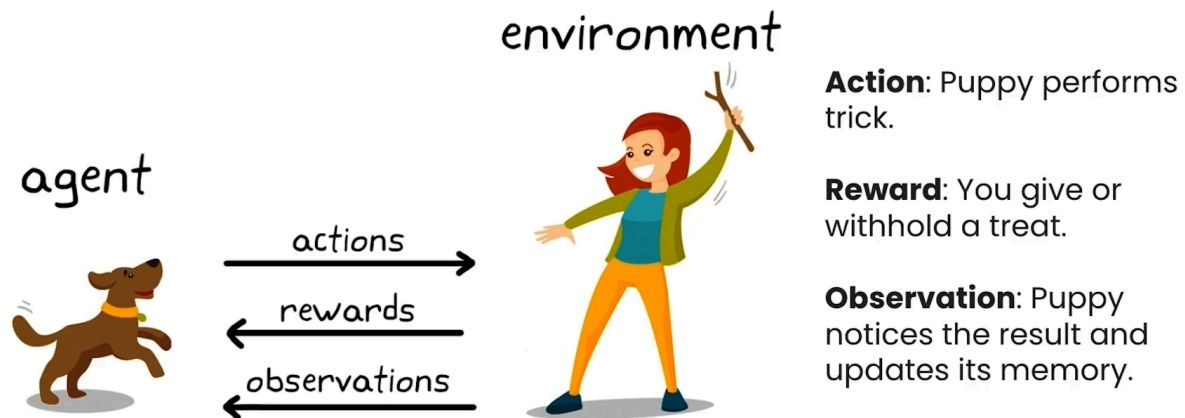
- Typically requires 1000s of labelled examples for the model to learn from - difficult and expensive to collect
- Overfitting - the model may learn patterns too specific to the training domain

These limitations suggest to find alternate approach that can reduce the reliance on extensive labeling and mitigate overfitting while still guiding the model towards the desired behavior.

One such alternative is  Reinforcement Learning where the model learns by interacting with its environment and optimizing for a reward signal rather than mimicking fixed labelled examples

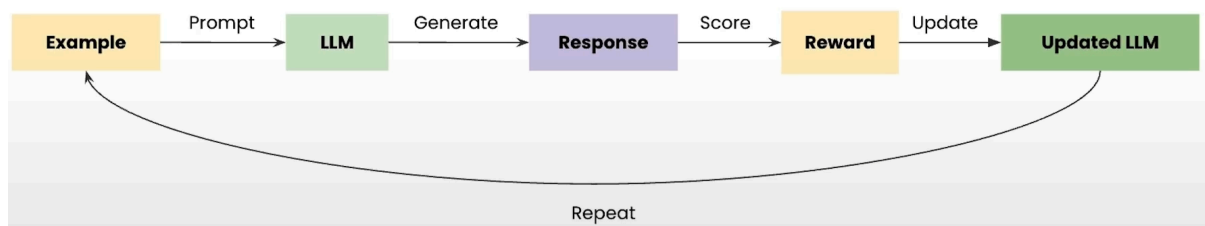
Reinforcement Learning

- Agent : Puppy
- Action: Fetch the stick, rollover, sit
- Reward: Treat on fetching the stick



Applying it to the LLMs

- Examples → prompt which comes from the environment → Feed it to the LLM → Takes action by generating a sequence of tokens as response → Evaluate the response using a score that will serve as the reward (human preference, quality, accuracy etc) → update weights to maximize rewards → Process can be repeated on new examples or the same ones

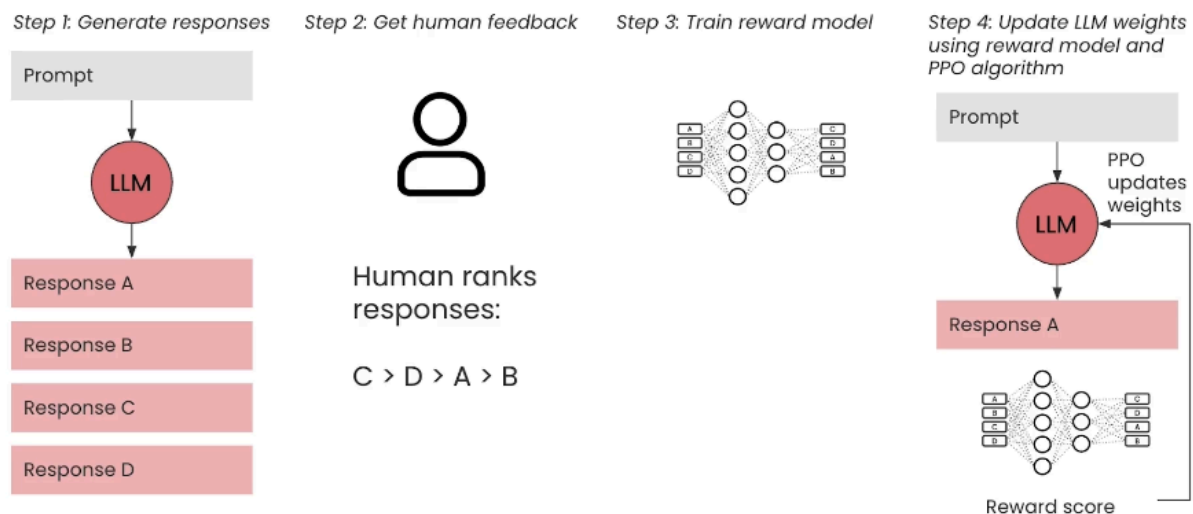


Reinforcement Learning with Human Feedback (RLHF).

This is the process that powers ChatGPT

1. **Generate responses:** Send prompt to LLM and sample multiple responses using temperature
2. **Get Human Feedback:** Ask annotators to rank the responses from best to worst

3. **Train Reward Model:** to learn to predict the human preferences. Prompt and response pair as input and output the score to indicate how good the response is
4. Finetune the LLM with each prompt using Proximal Policy Optimization (PPO).



Direct Preference Optimization (DPO)

instead of training a separate reward model, it finetunes the model on human preference pairs

1. Generate 2 responses
2. **Get Human Feedback:** Ask annotators to tell which response is more preferred (thumbs up/down)
3. **Create preference dataset:** Dataset containing prompt, chosen responses and the reject response
4. use the DPO algorithm to update the model weights that generates responses with high human preference. The idea is simple. For each prompt compare the probability distribution with that of preferred response or the rejected response and see which one it is more likely to generate.

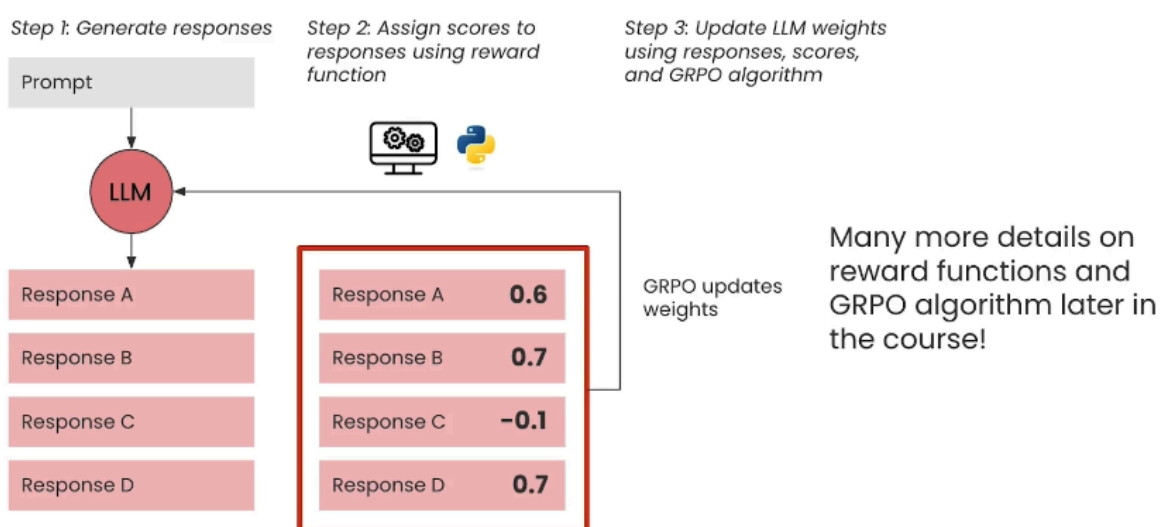
Both Reinforcement Learning with Human Feedback (RLHF) and Direct Preference Optimization (DPO) rely on human preferences instead of ground truth answers. But they differ in label format cost and risk.

Challenge	RLHF	DPO
Data Needed	Ranked generations for reward model	Paired-preference labels (A>B). But the data requirement is large
Compute/Memory	Very High	Moderate
Training Stability	Often unstable - reward hacking, collapse risk	more stable, but needs a lot of labels
Limitation	Doesn't teach new tasks - only steers preferences	Doesn't teach new tasks - only steers preferences

Group Relative Policy Optimization (GRPO)

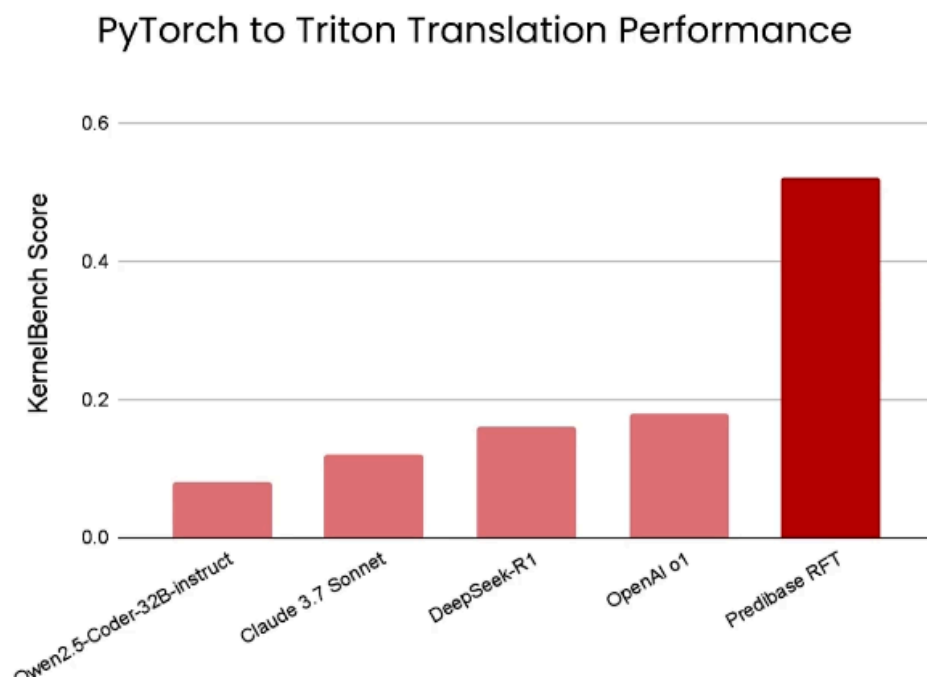
This is algorithm behind Deepseek -R1. This sidesteps the need for any human preference label by leaning on programmable reward functions that can be defined

1. Send prompt and sample multiple responses
2. Write programmable reward functions that assigns scores to responses
3. GRPO algorithm feeds each candidate's rewards as a training signal. It pushes up the probability of generating responses with above average scores in this group
4. Repeat the loop for GRPO to finetuning on the reward function directly without the need preference data



Benefits of Reinforcement Fine-tuning (RFT)

- It doesn't require labelled data. Just need a means to verify correctness
- Works with as few as 10 examples, but scales with increase in number of prompts that you show during training
- more flexible than SFT because it learns actively from feedback rather than fixed labelled examples
- Enables reasoning models to organically discover better strategies by improving its chain of thought



When to use RFT?

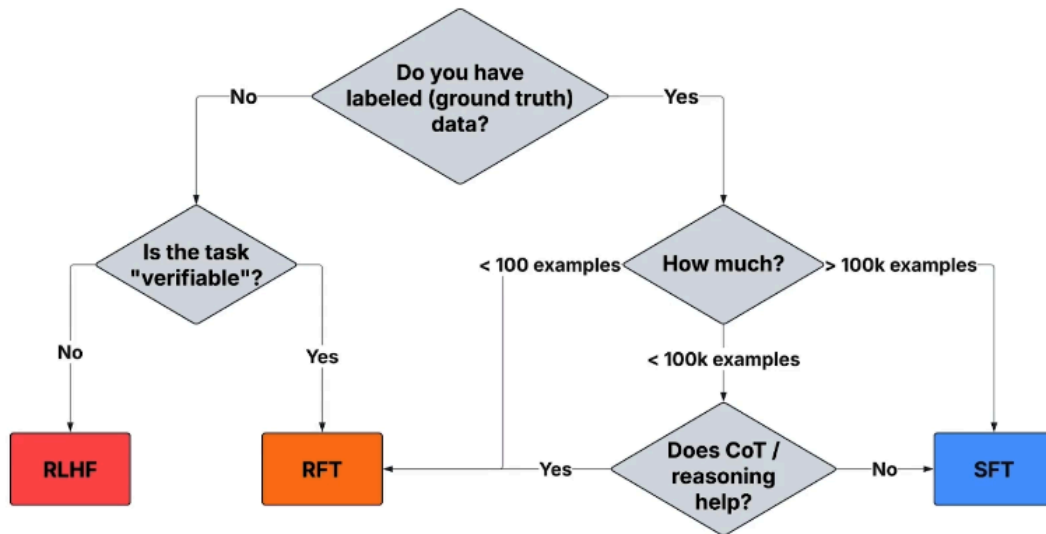
1. when you don't have labelled data - but can verify the correctness of the output produced
2. Limited labelled data for SFT
3. Chain of thought reasoning improves performance

Tasks suited for RFT

1. Mathematical problem solving - RFT provides detailed reasoning
2. Code generation and debugging - understanding the structure and intent of the code
3. Logical and multi-step reasoning - unlike pattern matching, when a task requires a sequence of decisions, reasoning models explain their decision

with step-by-step logic

When should you choose RFT over SFT?



Can LLM master wordle?

Wordle-style guessing game and textual feedback

Rules

- You have 6 tries to guess a secret 5-letter word
- Each guess must be valid
- After each guess, you will receive feedback indicating how close your guess was

Feedback Description	Wordle Color	Text Symbol
Correct letter and position		✓
Correct letter, wrong position		—
Letter not in the word		✗

[Lesson_3.html](#)

Reward Functions

1. Generate different responses from LLM
2. Score each response with a reward function and identify which responses is relatively better than others
3. Agent goal: maximize rewards received overtime
4. Two ingredients that are necessary for this learning to occur:
 - a. Diversity in response
 - b. Diversity in rewards
5. Calculate advantages to identify the relative desirability of responses

$$A_i = \frac{f_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

```
def compute_advantages(rewards: list):
    rewards = np.array(rewards)

    # Compute the mean and standard deviation of the rewards
    mean_reward = np.mean(rewards)
    std_reward = np.std(rewards)

    # Avoid division by zero in case of zero variance (typically happens when all rewards are 0)
    # Note: In the GRPO implementation, we add 1e-4 to the std_reward to avoid division by zero
    if std_reward == 0:
        return [0] * len(rewards)

    # Divide by stddev of rewards to normalize range to 0
    advantages = (rewards - mean_reward) / std_reward
    return advantages.tolist()
```


Reward function 1: Binary reward

If the guessed letter is correct then 1 else 0

```
def wordle_reward(guess: str, secret_word: str) -> int:
    if guess.upper() == secret_word.upper():
        return 1 # correct guess
    else:
        return 0 # incorrect guess
```

Advantages table from this reward function suggest that there is no relative advantage of predicting one word or the other despite some words have guess a letter in correction location

Secret: POUND

Index	Guess	Reward	Advantage
0	FOUNZ	0	0
1	OUTDO	0	0
2	TOUND	0	0
3	SOUND	0	0
4	SMILE	0	0
5	MOUNT	0	0
6	FROWN	0	0
7	BRAIN	0	0

Reward function with partial credit

```
def wordle_reward_partial_credit(guess: str, secret_word: str) -> float:
```

```

if len(guess) != len(secret_word):
    # no reward for having the wrong number of letters
    return 0.0

valid_letters = set(secret_word)
reward = 0.0
for letter, secret_letter in zip(guess, secret_word):
    if letter == secret_letter:
        # right letter, right location
        reward += 0.2
    elif letter in valid_letters:
        # right letter, wrong location
        reward += 0.1
    else:
        # no reward
        pass
return reward

```

Responses now have a relative advantage over other responses.

Temperature = 0.7

Secret: POUND

Index	Guess	Reward	Advantage
0	FOUND	0.8	1.3525
1	NOUDI	0.6	0.631169
2	FOUND	0.8	1.3525
3	WORD	0	-1.53284
4	CROWN	0.2	-0.811503
5	INDOM	0.3	-0.450835
6	DONUT	0.5	0.270501

+-----+-----+-----+-----+			
	7	FROWN	0.2 -0.811503
+-----+-----+-----+-----+			

Note: Temperature setting is important.

if Temperature = 0, then same word guessed again and again and would lead to no advantage

Secret: POUND

+-----+-----+-----+-----+			
	Index	Guess	Reward Advantage
+=====+=====+=====+=====+			
	0	DOWNY	0.5 0
+-----+-----+-----+-----+			
	1	DOWNY	0.5 0
+-----+-----+-----+-----+			
	2	DOWNY	0.5 0
+-----+-----+-----+-----+			
	3	DOWNY	0.5 0
+-----+-----+-----+-----+			
	4	DOWNY	0.5 0
+-----+-----+-----+-----+			
	5	DOWNY	0.5 0
+-----+-----+-----+-----+			
	6	DOWNY	0.5 0
+-----+-----+-----+-----+			
	7	DOWNY	0.5 0
+-----+-----+-----+-----+			

IF the temperature is very high = i.e. 1.3, however the responses are on average worse than the moderate temperature. Overall learning process will be slower because the guess quality is low. So set a reasonable temperature value close to 0.7

Secret: POUND

+-----+-----+-----+-----+			
	Index	Guess	Reward Advantage
+=====+=====+=====+=====+			

0	GROUND	0	-1.94198
1	SPIND	0.5	0.736614
2	DONOR	0.5	0.736614
3	TROND	0.5	0.736614
4	FINDS	0.2	-0.870544
5	NOSED	0.5	0.736614
6	RAINB	0.2	-0.870544
7	DONUT	0.5	0.736614

requirements.txt

utils.py

Lesson_4.html

Lesson_4.ipynb

Reward functions with LLM as Judge

Use LLM as a proxy for human judgement and create a reward functions learnings in situations where the outcomes are not easily verifiable